

CO5-AT1

CSA0613-Design & Analysis of Algorithms

Name:A.Bhavya(192424417)

Q3. Password Generation (Combinatorial Backtracking Problem) A system must generate secure passwords of length N with constraints such as at least one uppercase letter, one digit, and no repeated characters. Clearly define all constraints and input requirements. Analyze how these constraints influence the search space. Develop a backtracking-based solution that generates only valid passwords. Apply constraint checking and pruning techniques effectively. Justify how your solution ensures completeness and avoids invalid combinations..

AIM

To develop a backtracking-based solution for generating secure passwords of length N while satisfying constraints such as at least one uppercase letter, at least one digit, and no repeated characters.

OBJECTIVE

- To generate passwords of a specified length N .
- To ensure that every password contains at least one uppercase letter.
- To ensure that every password contains at least one digit.
- To avoid repeated characters in a password.
- To apply backtracking and pruning techniques.
- To generate only valid password combinations.

INTRODUCTION

Password generation is a combinatorial problem because a password can be formed using different combinations of characters. If all possible combinations are generated without restrictions, the search space becomes very large. Therefore, constraints are applied to generate secure and valid passwords.

Backtracking is an effective technique for this problem. It builds a password character by character and checks whether the current partial password can still satisfy all required conditions. If a partial password cannot produce a valid final password, that branch is stopped immediately. This process is called **pruning**.

INPUT REQUIREMENTS

The program requires the following input:

- **N** – Required length of the password.
- A set of lowercase letters.
- A set of uppercase letters.
- A set of digits.

For demonstration:

- Lowercase letters: abc
- Uppercase letters: ABC
- Digits: 123

The password length can be chosen by the user.

PASSWORD CONSTRAINTS

A valid password must satisfy the following conditions:

1. The password must have exactly **N characters**.
2. It must contain at least **one uppercase letter**.
3. It must contain at least **one digit**.
4. No character should be repeated.
5. Only characters from the allowed character set should be used.

SEARCH SPACE ANALYSIS

Suppose the total number of available characters is **M**.

Without considering the no-repetition condition, the total number of possible passwords of length N is:

$$M^n$$

When repeated characters are not allowed, the number of possible arrangements becomes:

$$M \times (M - 1) \times (M - 2) \times \dots$$

for N positions.

The uppercase and digit constraints further reduce the number of valid passwords.

Therefore, applying constraints and pruning helps avoid generating unnecessary combinations.

BACKTRACKING APPROACH

The password is constructed one character at a time.

At each step:

1. Select an unused character.
2. Add it to the current password.
3. Check the constraints.
4. Continue generating the next character.
5. If the current path cannot produce a valid password, stop exploring that path.
6. Remove the last character and try another character.

This process continues until the password reaches length N.

PRUNING TECHNIQUE

Pruning is used to reduce unnecessary searching.

For example, suppose only one position is left in the password, but the current password does not contain both an uppercase letter and a digit. Then it is impossible to satisfy both requirements using only one remaining position. Therefore, that branch is immediately stopped. This avoids generating invalid passwords and improves efficiency.

ALGORITHM

1. Start.
2. Input the password length N.
3. Define the allowed lowercase letters, uppercase letters, and digits.
4. Start with an empty password.
5. Select an unused character.
6. Add the character to the password.
7. Check whether the partial password can still satisfy all constraints.
8. If the constraints cannot be satisfied, prune the current branch.
9. If the password length becomes N, check whether it contains at least one uppercase letter and one digit.
10. If valid, display or store the password.
11. Remove the last character and try another unused character.
12. Repeat until all possible valid combinations are explored.
13. Stop.

PYTHON PROGRAM

main.py

Visualize

Run

```
1 def generate_password(password, used, n):
2
3     # Check if password reaches required length
4     if len(password) == n:
5
6         # Check required constraints
7         has_uppercase = any(char.isupper() for char in password)
8         has_digit = any(char.isdigit() for char in password)
9
10        if has_uppercase and has_digit:
11            print(password)
12
13        return
14
15    # Check remaining positions for pruning
16    remaining = n - len(password)
17
18    has_uppercase = any(char.isupper() for char in password)
19    has_digit = any(char.isdigit() for char in password)
20
21    missing = 0
22
```

main.py

Visualize

Run

```
1 def generate_password(password, used, n):
34     for char in characters:
37         if char not in used:
48             ,
49
50             # Backtracking
51             used.remove(char)
52
53     # Available characters
54     lowercase = "abc"
55     uppercase = "ABC"
56     digits = "123"
57
58     characters = lowercase + uppercase + digits
59
60     # Input password length
61     N = int(input("Enter password length: "))
62
63     print("\nValid Passwords:\n")
64
65     # Start backtracking
66     generate_password("", set(), N)
```

SAMPLE OUTPUT

```
Output 

Enter password length: 3

Valid Passwords:

aA1
aA2
aA3
aB1
aB2
aB3
aC1
aC2
aC3
a1A
a1B
a1C
a2A
a2B
a2C
a3A
a3B
a3C
```

EXPLANATION OF THE PROGRAM

The program uses recursion and backtracking to generate passwords.

A character is selected and added to the current password. The character is also stored in a used set to ensure that it is not selected again.

The program checks whether the current partial password can still satisfy the uppercase and digit requirements.

If there are not enough remaining positions to satisfy the missing requirements, the branch is pruned.

When the password reaches the required length, the program checks whether it contains:

- At least one uppercase letter.
- At least one digit.

Only valid passwords are displayed.

HOW THE SOLUTION ENSURES COMPLETENESS

The algorithm tries every possible unused character at every position.

Therefore, all possible character arrangements are explored.

A valid branch is never removed unless it is mathematically impossible for that partial password to satisfy the remaining constraints.

Hence, all valid passwords are generated.

HOW INVALID COMBINATIONS ARE AVOIDED

Invalid combinations are avoided using:

- A used set to prevent repeated characters.
- Uppercase checking.
- Digit checking.
- Pruning when remaining positions are insufficient.

This ensures that unnecessary invalid combinations are not explored completely.

TIME COMPLEXITY

Let M be the number of available characters.

Since characters cannot be repeated, the maximum number of arrangements is:

$$M \times (M - 1) \times \dots \times (M - N + 1)$$

Therefore, the algorithm has exponential/combinatorial growth.

The pruning technique reduces the practical search space by stopping invalid branches early.

SPACE COMPLEXITY

The recursion depth is equal to the password length N .

Therefore, the auxiliary space complexity is approximately:

$$O(N)$$

Additional space is used for storing generated passwords if they are stored instead of directly printed.

ADVANTAGES

- Generates passwords systematically.
- Ensures all required constraints are satisfied.
- Prevents repeated characters.
- Uses pruning to reduce unnecessary computation.
- Guarantees that all valid combinations are explored.

REAL-WORLD APPLICATIONS

This approach can be used in:

- Secure password generation systems.
- Authentication applications.
- Security testing.
- Combinatorial search problems.
- Constraint satisfaction systems.
- Test data generation.

CONCLUSION

The password generation problem was successfully solved using the backtracking technique. The algorithm constructs passwords character by character while checking the required constraints. The use of pruning reduces unnecessary exploration by stopping branches that cannot produce valid passwords. The used set ensures that no character is repeated. Thus, the solution generates all possible valid passwords of length N while ensuring at least one uppercase letter, one digit, and no repeated characters.

\